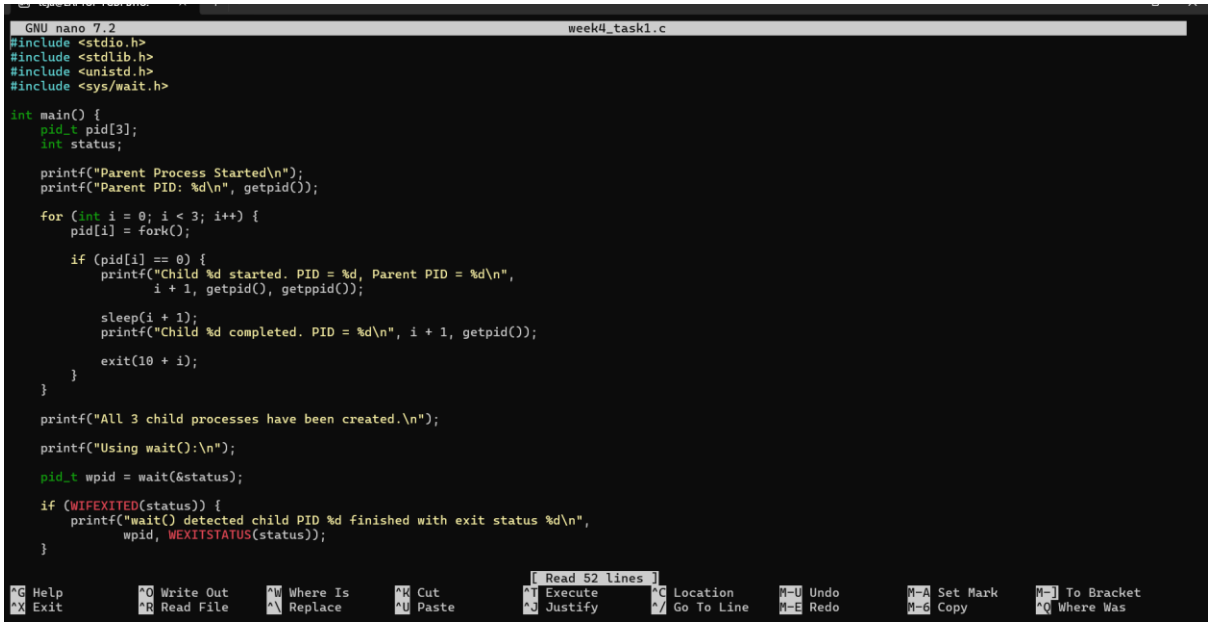
Task 1

Task 1

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions



```
 GNU nano 7.2                                  week4_task1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    for (int i = 0; i < 3; i++) {
        pid[i] = fork();

        if (pid[i] == 0) {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                   i + 1, getpid(), getppid());

            sleep(i + 1);
            printf("Child %d completed. PID = %d\n", i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("All 3 child processes have been created.\n");

    printf("Using wait():\n");

    pid_t wpid = wait(&status);

    if (WIFEXITED(status)) {
        printf("wait() detected child PID %d finished with exit status %d\n",
               wpid, WEXITSTATUS(status));
    }
                                [ Read 52 lines ]
^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-] To Bracket
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q Where Was
```

```
bhavan@LAPTOP-14JC0IVG:~$ cd ~
mkdir week4
cd week4
bhavan@LAPTOP-14JC0IVG:~/week4$ nano week4_task1.c
bhavan@LAPTOP-14JC0IVG:~/week4$ gcc week4_task1.c -o week4_task1
bhavan@LAPTOP-14JC0IVG:~/week4$ ./week4_task1
Parent Process: PID = 512
Child 1: PID = 513, Parent PID = 512

Child 2: PID = 514, Parent PID = 512
Parent: Waiting for any child using wait()...
Child 3: PID = 515, Parent PID = 512
Child 3 completed.
wait(): Child PID 515 completed with exit status 30

Parent: Waiting specifically for Child 1 using waitpid()...
Child 2 completed.
Child 1 completed.
waitpid(): Child 1 PID 513 completed with exit status 10
waitpid(): Child 2 PID 514 completed with exit status 20

Parent Process: All child processes completed.
bhavan@LAPTOP-14JC0IVG:~/week4$ nano week4_task2.c
bhavan@LAPTOP-14JC0IVG:~/week4$ gcc week4_task2.c -o week4_task2
bhavan@LAPTOP-14JC0IVG:~/week4$ ./week4_task2
Parent: PID = 530
Parent: Child PID = 531
Parent: Sleeping for 30 seconds...
Child: PID = 531
Child: Exiting now...
Parent: Exiting...
```

```
Parent: Exiting...
bhavan@LAPTOP-14JC0IVG:~/week4$ ps -el
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S     0       1       0  0  80   0 -  5414 -      ?        00:00:00 systemd
5 S     0       2       1  0  80   0 -   795 -      hvc0     00:00:00 init-systemd(Ub
0 S     0       7       2  0  80   0 -   799 -      hvc0     00:00:00 init
4 S     0      56       1  0  79  -1 - 12590 -      ?        00:00:00 systemd-journal
4 S     0     106       1  0  80   0 -  6316 -      ?        00:00:00 systemd-udevd
4 S   991     115       1  0  80   0 -  5365 -      ?        00:00:00 systemd-resolve
4 S   996     116       1  0  80   0 - 22757 -      ?        00:00:00 systemd-timesyn
4 S     0     180       1  0  80   0 -  1061 -      ?        00:00:00 cron
4 S   101     181       1  0  80   0 -  2399 -      ?        00:00:00 dbus-daemon
4 S     0     194       1  0  80   0 -  4493 -      ?        00:00:00 systemd-logind
4 S     0     196       1  0  80   0 - 439026 -     ?        00:00:00 wsl-pro-service
4 S   102     207       1  0  80   0 - 55629 -      ?        00:00:00 rsyslogd
4 S     0     218       1  0  80   0 -   781 -      tty1     00:00:00 agetty
4 S     0     231       1  0  80   0 - 26754 -      ?        00:00:00 unattended-upgr
5 S     0     313       2  0  80   0 -   796 -      ?        00:00:00 SessionLeader
5 S     0     314     313  0  80   0 -   800 -      ?        00:00:00 Relay(315)
4 S  1000     315     314  0  80   0 -  1520 do_wai pts/0    00:00:00 bash
4 S     0     316       2  0  80   0 -  1676 -      pts/1    00:00:00 login
4 S  1000     360       1  0  80   0 -  5079 -      ?        00:00:00 systemd
5 S  1000     361     360  0  80   0 -  5288 -      ?        00:00:00 (sd-pam)
4 S  1000     387     316  0  80   0 -  1520 poll_s pts/1    00:00:00 bash
0 R  1000     538     315  0  80   0 -  2080 -      pts/0    00:00:00 ps
```

```
bhavan@LAPTOP-14JC0IVG:~/week4$ history
    1  (user "root"), use "sudo <command>".
    2  "man sudo_root"
    3  sudo apt update
    4  pass change
    5  sudo apt update
    6  password change
    7  pass
    8  sudo apt update
    9  sudo apt install tree
   10  reset password
   11  sudo apt install
   12  wsl --install
   13  sudo apt update
   14  sudo apt install bulid-essential
   15  gcc --version
   16  sudo apt install gcc
   17  gcc --version
   18  sudo apt install gcc
   19  gcc --version
   20  nano hello.c
   21  gcc hello.c -o hello
   22  ./hello
   23  nano hello.c
   24  ls
   25  pwd
   26  date
   27  whoami
   28  hostname
   29  uname
   30  uname -r
   31  id
   32  tty
   33  uptime
   34  users
   35  who
   36  w
   37  groups
   38  env
   39  printenv
   40  df
```

```
41  free
42  ps
43  top
44  vmstat
45  lsblk
46  lscpu
47  mount
48  history
49  clear
50  bc
51  echo hi
52  factor 456
53  yes
54  mkdir OSSP
55  cd OSSP
56  git init
57  mkdir src include obj bin docs tests scripts assets
58  touch .gitignore README.md LICENSE Makefile
59  touch src/main.c
60  touch src/shell.c
61  touch src/parser.c
62  touch src/executor.c
63  touch src/builtin.c
64  touch include/shell.h
65  touch include/parser.h
66  touch include/executor.h
67  touch include/builtin.h
68  touch docs/design.md
69  touch docs/report.pdf
70  touch tests/test_parser.c
71  touch tests/test_executor.c
72  touch scripts/run.sh
73  touch assets/architecture.png
74  tree
75  sudo apt install tree
76  tree
77  cd ~/OSSP
78  nano command_executor.c
79  gcc command_executor.c -o command_executor
80  ./command_executor
81  gcc process.c -o process
```

```
 81  gcc process.c -o process
 82  ./process
 83  gcc process.c -o process
 84  gcc --version
 85  nano process.c
 86  gcc process.c -o process
 87  ./process
 88  ls
 89  pwd
 90  date
 91  cal
 92  whoami
 93  hostname
 94  uname
 95  uname -a
 96  ps
 97  history
 98  lscpu
 99  ps
100  ps -ef
101  history
102  mkdir OSSP
103  cd OSSP
104  mkdir OSSP
105  cd OSSP
106  git init
107  mkdir src include obj bin docs tests scripts assets
108  touch src/main.c
109  touch src/shell.c
110  touch src/parser.c
111  touch src/executor.c
112  touch src/builtin.c
113  Install MongoDB Compass (Standalone)
114  cd ~
115  mkdir week4
```

```
120  nano week4_task2.c
121  gcc week4_task2.c -o week4_task2
122  ./week4_task2
123  ps -el
124  Z
125  ps -o pid,ppid,state,cmd -p 159
126  ls
127  pwd
128  cat week4_task1.c
129  cat week4_task2.c
130  ps aux | grep week4
131  clear
132  history
bhavan@LAPTOP-14JC0IVG:~/week4$
```

A parent process can create multiple child processes using the fork() system call. After creating the child processes, the parent may need to wait for them to complete before continuing.

The wait() function allows the parent process to wait until any one child process finishes. The waitpid() function gives more control because the parent can wait for a specific child process.

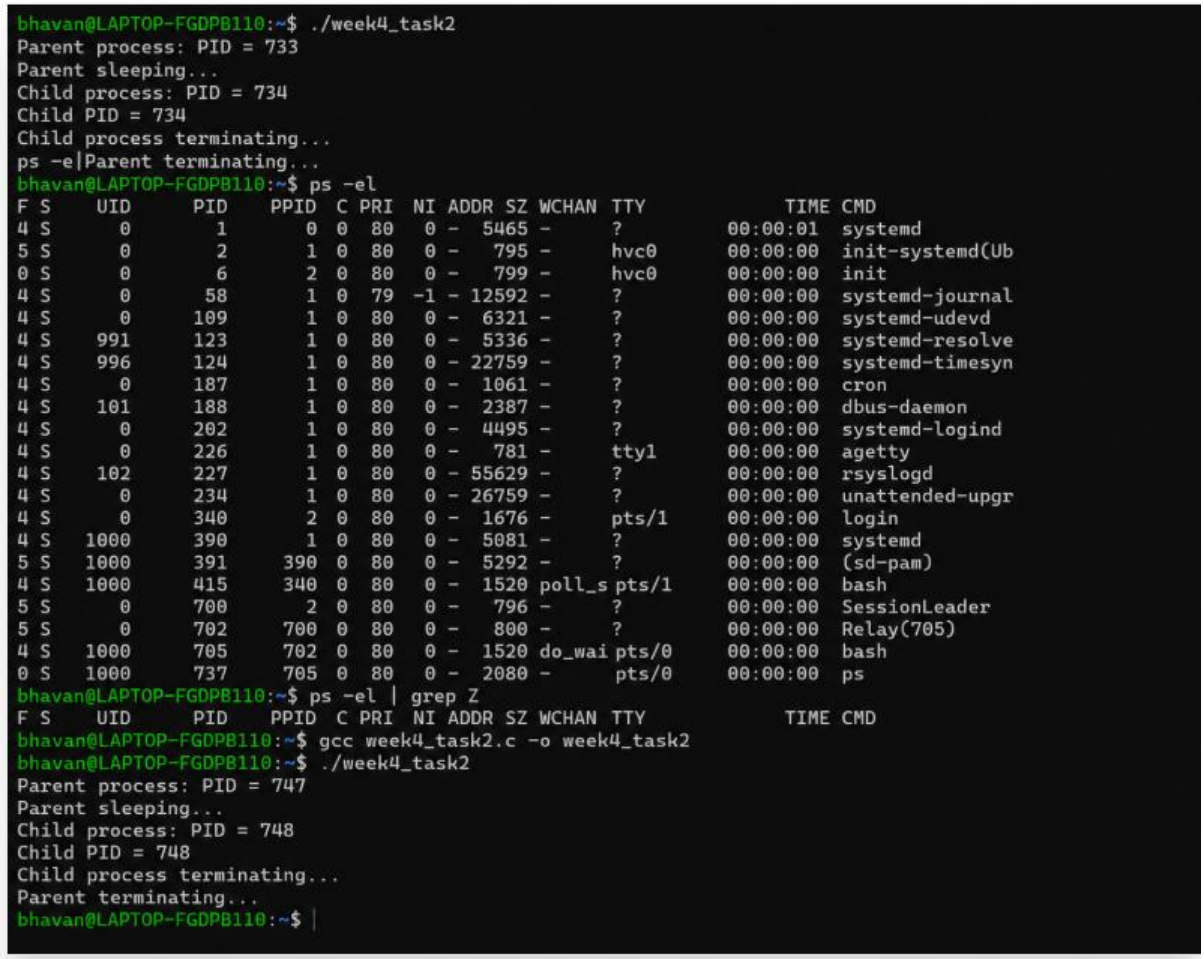
In this practical, three child processes are created. The parent uses wait() and waitpid() to synchronize their completion and identify when each child process has finished

Task 2 Question

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child PID = %d\n", getpid());
        printf("Child process terminating...\n");
        exit(0);
    }
    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Parent sleeping...\n");

        sleep(10);

        wait(NULL);

        printf("Parent terminating...\n");
    }

    return 0;
}
```

```
bhavan@LAPTOP-FGDPB110:~$ nano week4_task2.c
bhavan@LAPTOP-FGDPB110:~$ gcc week4_task2.c  -o week4_task2
bhavan@LAPTOP-FGDPB110:~$ ./week4_task2
Parent process: PID = 733
Parent sleeping...
Child process: PID = 734
Child PID = 734
Child process terminating...
ps -elParent terminating...
bhavan@LAPTOP-FGDPB110:~$ ps -el
F S   UID     PID    PPID C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S     0       1       0 0  80   0 -  5465 -      ?        00:00:01 systemd
5 S     0       2       1 0  80   0 -   795 -      hvc0     00:00:00 init-systemd(Ub
0 S     0       6       2 0  80   0 -   799 -      hvc0     00:00:00 init
4 S     0      58       1 0  79  -1 - 12592 -      ?        00:00:00 systemd-journal
4 S     0     109       1 0  80   0 -  6321 -      ?        00:00:00 systemd-udevd
4 S   991     123       1 0  80   0 -  5336 -      ?        00:00:00 systemd-resolve
4 S   996     124       1 0  80   0 - 22759 -      ?        00:00:00 systemd-timesyn
4 S     0     187       1 0  80   0 -  1061 -      ?        00:00:00 cron
4 S   101     188       1 0  80   0 -  2387 -      ?        00:00:00 dbus-daemon
4 S     0     202       1 0  80   0 -  4495 -      ?        00:00:00 systemd-logind
4 S     0     226       1 0  80   0 -   781 -      tty1     00:00:00 agetty
4 S   102     227       1 0  80   0 - 55629 -      ?        00:00:00 rsyslogd
4 S     0     234       1 0  80   0 - 26759 -      ?        00:00:00 unattended-upgr
4 S     0     340       2 0  80   0 -  1676 -      pts/1    00:00:00 login
4 S  1000     390       1 0  80   0 -  5081 -      ?        00:00:00 systemd
5 S  1000     391     390 0  80   0 -  5292 -      ?        00:00:00 (sd-pam)
4 S  1000     415     340 0  80   0 -  1520 poll_s pts/1    00:00:00 bash
5 S     0     700       2 0  80   0 -   796 -      ?        00:00:00 SessionLeader
5 S     0     702     700 0  80   0 -   800 -      ?        00:00:00 Relay(705)
bhavan@LAPTOP-FGDPB110:~$
```



THEORY

A **zombie process** is a child process that has finished its execution, but its parent process has not yet collected its exit status. Because of this, the child remains in the process table.

In this practical, the child process terminates while the parent process is sleeping. The process table can be checked using the ps -el command, where the zombie process appears with the state **Z**.

To remove the zombie process, the parent uses the wait() system call. This collects the child's exit status and allows the system to remove the child from the process table.

```
teja@LAPTOP-FGDPB11O:~$ ./week4_task2
Parent process: PID = 733
Parent sleeping...
Child process: PID = 734
Child PID = 734
Child process terminating...
ps -elParent terminating...
teja@LAPTOP-FGDPB11O:~$ ps -el
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S     0       1       0  0  80   0 -  5465 -      ?        00:00:01 systemd
5 S     0       2       1  0  80   0 -   795 -      hvc0     00:00:00 init-systemd(Ub
0 S     0       6       2  0  80   0 -   799 -      hvc0     00:00:00 init
4 S     0      58       1  0  79  -1 - 12592 -      ?        00:00:00 systemd-journal
4 S     0     109       1  0  80   0 -  6321 -      ?        00:00:00 systemd-udevd
4 S   991     123       1  0  80   0 -  5336 -      ?        00:00:00 systemd-resolve
4 S   996     124       1  0  80   0 - 22759 -      ?        00:00:00 systemd-timesyn
4 S     0     187       1  0  80   0 -  1061 -      ?        00:00:00 cron
4 S   101     188       1  0  80   0 -  2387 -      ?        00:00:00 dbus-daemon
4 S     0     202       1  0  80   0 -  4495 -      ?        00:00:00 systemd-logind
4 S     0     226       1  0  80   0 -   781 -      tty1     00:00:00 agetty
4 S   102     227       1  0  80   0 - 55629 -      ?        00:00:00 rsyslogd
4 S     0     234       1  0  80   0 - 26759 -      ?        00:00:00 unattended-upgr
4 S     0     340       2  0  80   0 -  1676 -      pts/1    00:00:00 login
4 S  1000     390       1  0  80   0 -  5081 -      ?        00:00:00 systemd
5 S  1000     391     390  0  80   0 -  5292 -      ?        00:00:00 (sd-pam)
4 S  1000     415     340  0  80   0 -  1520 poll_s pts/1    00:00:00 bash
5 S     0     700       2  0  80   0 -   796 -      ?        00:00:00 SessionLeader
5 S     0     702     700  0  80   0 -   800 -      ?        00:00:00 Relay(705)
4 S  1000     705     702  0  80   0 -  1520 do_wai pts/0    00:00:00 bash
0 R  1000     737     705  0  80   0 -  2080 -      pts/0    00:00:00 ps
teja@LAPTOP-FGDPB11O:~$ ps -el | grep Z
F S   UID     PID    PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
teja@LAPTOP-FGDPB11O:~$ gcc week4_task2.c -o week4_task2
./week4_task2
Parent process: PID = 747
Parent sleeping...
Child process: PID = 748
Child PID = 748
Child process terminating...
Parent terminating...
teja@LAPTOP-FGDPB11O:~$
```